



OpenCV

💡 OpenCV를 배우는 이유

" 컴퓨터의 '눈'을 만드는 기술 "

컴퓨터는 우리처럼 사물들을 보고 곧 바로 "장애물이다!" 라고 인식할 수 없다.
카메라 센서를 통해서 얻어야 하는 것들을 컴퓨터에게 알려주기 위해 OpenCV를 사용하는 것이다.

+ 수많은 비전 라이브러리 중에서도 속도가 매우 빨라, 실시간 로봇 제어나 자율 주행에 최적화되어 있다.

🔄 전체 흐름

1. 테스트 주행 이미지 자동 생성
2. Gray 변환 (그레이스케일)
3. Gaussian Blur
4. Canny Edge Detection
5. ROI 마스크 생성
6. ROI 내부 에지만 추출
7. Hough Line Transform으로 직선 검출
8. 검출된 선을 원본 이미지에 표시
9. ROS2 주행 판단 관점으로 해석

단순 윤곽선을 추출하기 전에 흑백, 블러 처리로 전처리를 해주고, 연산속도를 빠르게 하기 위해 ROI(관심영역)을 설정해주고, 후처리 후 자율주행에 적용하는 것이다.

Canny Edge를 하는 것들

차선 경계 검출
물체 외곽선 추출
벽/장애물 경계 확인
문턱/라인 인식
마커 후보 영역 탐색
SLAM 전처리 아이디어 이해

💡 Edge란 무엇인가?

흰색 배경 → 검은색 사각형
밝기 255 → 밝기 0

이렇게 값이 급격하게 바뀌는 곳이 Edge 이다.

즉, 쉽게 얘기하면 "물체의 경계 후보선" 이다.

Canny Edge 기본 함수

```
edges = cv2.Canny(gray_or_blur, threshold1, threshold2)

# threshold1 : 약한 엣지값 / threshold2 : 강한 엣지값
# 이 threshold 값이 낮으면 많은 후보들 보여주고, 높으면 확실한 선들만 표시.
```

ROI 영역 설정

하늘, 벽, 천장, 배경, 사람, 멀리 있는 물체

보통 자율주행에 필요하지 않은 것들의 영역을 설정해서 없애버리기 위함이다. + 메모리 관리

전처리부터 ROI 영역 설정 후 선 따는 핵심 코드 15가지

```
# 1. 검은색 빈 배경 생성: (높이, 너비, 채널) 순서로 캔버스 준비 또는 카메라로 받아오는.
image = np.full((480, 640, 3), 70, dtype=np.uint8)

# 2. 컬러 변환 (BGR -> Gray): 연산 속도 향상을 위해 1채널로 압축
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# 3. 가우시안 블러: 노이즈를 제거하여 부드러운 이미지 생성
blurred = cv2.GaussianBlur(gray, (5, 5), 0)

# 4. 캐니 에지 검출: 픽셀 값의 차이가 큰 경계선(선 후보) 추출
edges = cv2.Canny(blurred, 50, 150)

# 5. 마스크 도화지 생성: 에지 이미지와 똑같은 크기의 검은 화면 준비
mask = np.zeros_like(edges)

# 6. ROI 다각형 좌표 설정: 차선이 존재하는 사다리꼴 영역 지정
polygon = np.array([[ (80, 480), (260, 260), (380, 260), (580, 480) ]], dtype=np.int32)

# 7. 다각형 채우기: 마스크의 ROI 영역만 흰색(255)으로 색칠
cv2.fillPoly(mask, polygon, 255)

# 8. 비트 연산 (AND): 에지 이미지와 마스크를 겹쳐 관심 영역만 추출
roi_edges = cv2.bitwise_and(edges, mask)

# 9. 허프 변환 직선 검출: 점들을 연결해 실제 직선의 좌표(x1, y1, x2, y2) 계산
lines = cv2.HoughLinesP(roi_edges, 1, np.pi/180, 45, minLineLength=40, maxLineGap=35)

# 10. 좌표 언패킹: 검출된 선 리스트에서 개별 좌표를 추출
for line in lines:
    x1, y1, x2, y2 = line[0]

# 11. 원본에 선 그리기: 인식된 차선을 원본 이미지 위에 표시
cv2.line(result, (x1, y1), (x2, y2), (0, 0, 255), 4)
```

```
# 12. 다각형 외곽선 그리기: 설정한 ROI 영역이 어디인지 시각적으로 확인
cv2.polylines(result, polygon, True, (0, 255, 0), 3)

# 13. 중심점 계산: 검출된 선들의 평균 위치를 구해 차량의 현재 위치 파악
avg_x = sum(x_centers) / len(x_centers)

# 14. 텍스트 삽입: 이미지 위에 판단 결과나 데이터 수치를 출력
cv2.putText(result, "Detected Lines", (30, 440), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 2)

# 15. 이미지 파일 저장: 도커 환경에서 결과를 확인하기 위한 최종 단계
cv2.imwrite('/workspace/outputs/result.png', result)
```